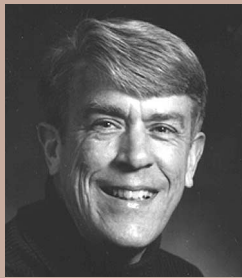


Questioning the Software Engineering Unquestionables

Robert L. Glass

... in which I oppose the software field's tendency to look at all new concepts uncritically

recently reviewed yet another book in the somehow burgeoning series on Extreme Programming. (Why does a lean and mean methodology call for so many books to describe it?) But this one was completely different.



The book is Pete McBreen's *Questioning Extreme Programming* (Addison-Wesley, 2003). And question he does. This is not just another me-too, oh-how-great-it-is book; it genuinely questions the wisdom of using XP on any of your software projects.

What a great concept! I'd like that concept to broaden into a software engineering way of life. It is high time that we in the software field questioned some of its old wives' and husbands' tales. That we question the hype spewing from the hype purveyors, especially vendors. That we question the advocacy spewing from all too many computer science researchers, who seem to feel that research papers should "conceive of a concept, advocate the concept, and scold practitioners who refuse to use the concept."

Imagine the outcome of such questioning. We could have books with titles such as

- *Questioning CASE Tools*
- *Questioning Object Orientation*
- *Questioning Web Services*
- *Questioning Formal Methods*

And we could engage in honest-to-goodness, objective debates about the merits of all those new concepts that keep flowing down the software engineering pipeline. The result could be something like what I'm trying to achieve in this department. No new concept could be seriously entertained without passing through a "questioning" sieve.

One size does not fit all

Just to flesh out this idea, let me describe how McBreen instantiates it in his book. He begins with a 55-page chapter that defines XP and talks in general about his outlook on methodologies. (I don't intend to replicate that definition here. If you don't know what XP is about, read the first and best book on the subject, Kent Beck's *Extreme Programming Explained* [Addison-Wesley, 1999]).

It is worth spending a minute on how McBreen sees methodologies, because that colors his whole view of XP. He believes that "all processes are situational," that seeking one-size-fits-all methodologies is doomed to failure,

■ *Questioning Structured Programming*

Continued on p. 119

Continued from p. 120

and that “software is not a mechanical process and ... you will never be able to adopt a process without doing some adapting to fit ... your circumstances and team.” Given that position, of course, no methodology—including XP—will completely pass his appropriateness test.

Furthermore, he points out that no one has performed comparative studies of XP use on reasonable-sized projects. He then notes wryly that no other methodologies have been subjected to such evaluative research, either. (Computing researchers, are you listening?) Given all that, until researchers change their ways, all we have to judge your typical methodology—including XP—is the hopefully objective views of those who do that questioning I mentioned earlier.

So, given that the field of questioning methodologies in general and XP in particular is dismal, what *can* be said about XP? McBreen uses a nice approach. He talks about each element of XP (I like to call that “disaggregating” the methodology), explains why XP advocates laud that element, and then presents a collection of “open questions” about those advocacy claims. For example, regarding the XP requirement for on-site customer presence during project development, one open question is, “Is it easy to find a customer who speaks for all the voices of the customer/user organization?” Another is, “Can such a customer really work at a sustainable pace during an XP project?” (That is, will such a customer separated from his or her peers have enough to do during project development?)

More than just criticism

But this is not just an anti-XP book. In each chapter, McBreen answers the question “What can other approaches learn from XP?” He might be questioning XP’s value, but he is also questioning the alternatives’ value and looking for ways to meld the best of all approaches.

Later, McBreen gets even more spe-

cific. For each XP concept, he also poses such questions as “Why now?” (that is, why did this concept arise at this point in the history of software engineering?) and “What does this mean for XP?” (how does this concept blend with the other parts of the methodology?).

The bottom line

Given all of that, you might be interested in the bottom line McBreen gives in his questioning of XP. He sees the approach as having only a “very narrow applicability.” Specifically, he sees XP as best suited for projects with a narrow range of team sizes, from four to 12 programmers. It is totally unsuited for

- Life- or safety-critical embedded systems
- Larger team sizes
- Tiny teams (with one to two programmers on a month-long project)

McBreen also poses a set of questions to ask if you are thinking of using XP in your organization:

- Is my current approach broken?
- Is my organization ready for XP?
- Do my developers want to use it?
- Are our projects suitable (given the earlier discussion)?

Only a set of “yes” answers, he says, should cause you to consider XP.

I would love to see a similar analysis of some of the other new software engineering ideas that come along. For example, I would love to know for which projects object orientation works best and for which it is inappropriate. (I refuse to believe that OO, or any other methodology, is truly universal. And if so, then precisely when should we use OO—or the structured approaches, or formal methods, or Web Services—and when shouldn’t we?)

We still don’t understand many things about the software engineering field—things that we cannot understand until we take a more questioning look at what we are doing. I laud Pete McBreen for questioning the unquestionables about XP. I hope his is only the first in a long succession of “Questioning” books. After all, that would seem a more appropriate series to pursue than another Idiot’s or Dummies collection! ☺

Robert L. Glass is editor emeritus of Elsevier’s *Journal of Systems and Software*, the publisher and editor of the *Software Practitioner* newsletter, and a certified “premier curmudgeon of software practice.” Contact him at rglass@indiana.edu; he’d be pleased to hear from you.

Copyright and reprint permission: Copyright © 2003 by the Institute of Electrical and Electronics Engineers, Inc. All rights reserved. Abstracting is permitted with credit to the source. Libraries are permitted to photocopy beyond the limits of US copyright law for private use of patrons those post-1977 articles that carry a code at the bottom of the first page, provided the per-copy fee indicated in the code is paid through the Copyright Clearance Center, 222 Rosewood Dr., Danvers, MA 01923. For copying, reprint, or republication permission, write to Copyright and Permissions Dept., IEEE Publications Admin., 445 Hoes Ln., Piscataway, NJ 08855-1331.

Circulation: *IEEE Software* (ISSN 0740-7459) is published bimonthly by the IEEE Computer Society. IEEE headquarters: Three Park Ave., 17th Floor, New York, NY 10016-5997. IEEE Computer Society Publications Office: 10662 Los Vaqueros Cir., PO Box 3014, Los Alamitos, CA 90720-1314; (714) 821-8380; fax (714) 821-4010. IEEE Computer Society headquarters: 1730 Massachusetts Ave. NW, Washington, DC 20036-1903. Subscription rates: IEEE Computer Society members get the lowest rates and choice of media option—\$43/34/56 US print/electronic/combination; go to <http://computer.org/subscribe> to order and for more information on other subscription prices. Back issues: \$10 for members, \$20 for nonmembers (plus shipping and handling). This magazine is available on microfiche.

Postmaster: Send undelivered copies and address changes to Circulation Dept., *IEEE Software*, PO Box 3014, Los Alamitos, CA 90720-1314. Periodicals Postage Paid at New York, NY, and at additional mailing offices. Canadian GST #125634188. Canada Post Publications Mail Product (Canadian Distribution) Sales Agreement Number 0487805. Printed in the USA.

Reproduced with permission of the copyright owner. Further reproduction prohibited without permission.